SELECTIVE ADVERSARIAL SEARCH FOR CHESS UNDER TIME CONSTRAINTS

ABSTRACT

A chess engine must compare adversarial continuations within a limited time budget. Its decisions depend on both the positions it examines and the values assigned to its search frontier. We describe a classical search method that combines negamax backups with alpha–beta pruning, a phase-dependent positional evaluation, and tactical continuation at the nominal horizon. Iterative deepening maintains a completed decision as search proceeds. Move ordering and transposition reuse reduce repeated work, while bounded extensions and late move reductions allocate depth according to tactical relevance. An opening book provides a separate decision rule for covered early positions. We give a common mathematical formulation of these components and identify the assumptions under which pruning preserves a fixed-tree minimax value. A desktop deployment connects this procedure to a live game through asynchronous search, clock-aware budgets, and validation of returned actions. Annotated game states and an execution trace illustrate the interaction between the search rules and the deployed agent.

1 INTRODUCTION

Chess makes the cost of deliberation explicit. Every legal move permits a set of opponent replies, each of which creates further choices. A useful decision therefore requires an estimate of what both players can achieve along a continuation. Exhaustive analysis becomes infeasible as the horizon grows, so a practical engine must decide where to stop, how to value the resulting positions, and which parts of the tree deserve additional computation.

The classical approach couples a search procedure with a positional evaluation (Shannon, 1950). Search exposes consequences that are difficult to recognize from the current board alone. Evaluation supplies information about continuations beyond the examined horizon. These components interact: an immediate material gain may disappear after a recapture, and a forcing sequence may remain unresolved when a uniform depth limit is reached. The choice of search frontier is consequently part of the decision method.

Approach and scope. We study this interaction in a chess engine with fixed evaluation parameters and a finite deliberation budget. The engine searches in the negamax form of minimax. Alpha–beta bounds eliminate continuations that cannot affect an ancestor's decision, and tactical search refines frontier positions. Depth is allocated selectively through extensions and reductions. Search results also inform move ordering and can be reused when the same position is encountered again. Iterative deepening connects these mechanisms to a deadline by retaining the action from the latest completed root search.

We formulate an integration of established techniques at the level of states, values, search bounds, and resource allocation. The formulation specifies how the positional evaluator interacts with the tactical frontier and identifies the scope of the fixed-tree pruning guarantee. We then describe how this decision procedure operates within a desktop game, including the transfer of game state, interruption of search, and validation of a returned action. Captured positions and worker messages make these interactions observable. Playing strength and the benefit of individual search enhancements require separate evaluation.

2 RELATED WORK

Classical chess search. Shannon (1950) formulated computer chess around the combination of look-ahead search and position evaluation. Knuth and Moore (1975) analyzed alpha–beta pruning and the dependence of its cost on move ordering. Schaeffer (1989) examined the history heuristic together with other search enhancements, including transposition tables and killer moves. These techniques provide the search foundation used here. Their interaction matters because a promising move examined early can establish a bound that avoids substantial work later in the same search.

Selective analysis and evaluation. The design of Deep Blue illustrates the importance of nonuniform search, extensions, quiescence analysis, and structured chess knowledge (Campbell et al., 2002). These mechanisms address different sources of error: evaluation estimates positional value, while additional search resolves concrete tactical consequences. Our formulation uses a compact collection of material, placement, pawn, king, and center terms, together with a bounded selective search. The description concerns the roles of these components and the decision rules that connect them.

Learning and search. AlphaZero combines self-play learning with neural policy and value functions and Monte-Carlo tree search (Silver et al., 2018). Grill et al. (2020) interpret its action-selection mechanism through regularized policy optimization. These works show how a search procedure can operate with learned estimates and action preferences. In the present engine, evaluation parameters are prescribed, action preferences come from search statistics and chess features, and values are propagated by adversarial negamax backups. The common issue is how a limited search budget is allocated across candidate continuations.

3 Problem Formulation

Game model. Let $\mathcal{S}$ be the set of legal game states. A state s contains the board, the player to move $c(s)$, castling and en-passant rights, and the history needed to determine draw conditions. We write $\bar{c}$ for the opponent of player c, $\mathcal{A}(s)$ for the legal actions, and $T(s, a)$ for the deterministic successor. Each transition changes the player to move. A *ply* is one such transition.

Let $\mathcal{Z} \subset \mathcal{S}$ denote terminal states. Search assigns a draw value of zero and a checkmate value from the perspective of the player to move:

$$\tau(s, \ell) = \begin{cases} -M + \ell, & \text{if the player to move is checkmated,} \\ 0, & \text{if the search recognizes a draw,} \end{cases} \qquad s \in \mathcal{Z}, \tag{1}$$

where ℓ is the distance from the current root. For the static evaluator E_θ defined in Section 4.1, choose $M - L_{\max} > \sup_{s \notin \mathcal{Z}} |E_\theta(s)|$ so that mate values dominate heuristic scores. Negating the terminal value along a path gives a preference for an earlier forced win and a later forced loss. Draw detection covers stalemate, insufficient material, repetition, and the move-count rule adopted by the engine.

Decision objective. For a nonterminal state, the decision procedure receives a time allowance B and a maximum nominal depth $d_{\max}$. Its output is a legal action $\hat{a} \in \mathcal{A}(s)$. The underlying game objective is the zero-sum terminal outcome. Within the available budget, the engine approximates that objective through a finite tree whose frontier is scored by a heuristic evaluator. An evaluation score expresses relative positional preference; it carries no calibrated interpretation as a probability of winning.

4 Search Framework

The decision procedure has three stages. It first checks whether an eligible opening-book action is available. Otherwise, it performs root searches at increasing nominal depths. Each search combines bounded adversarial backups with positional evaluation and tactical continuation at the frontier. Algorithm 1 gives the resulting budgeted decision rule.

4.1 Positional Evaluation

For player c, let $F_\theta(s, c)$ measure the strength of that player's position. The parameter collection θ contains fixed piece values, positional tables, and feature weights. We decompose the score as

$$F_\theta(s, c) = M_c(s) + L_c(s) + P_c(s) + K_c(s) + C_c(s) + W_c(s). \tag{2}$$

The terms represent material, piece placement, pawn structure, king safety, central control, and endgame conversion. All use a common score scale. Table 1 summarizes the information they encode.

The parameters are specified before play and shared across searched positions. Position-dependent features, including the phase variables, are recomputed as moves change the board. Search statistics alter exploration order without updating θ. The resulting score expresses the current arrangement under a fixed set of positional preferences.

Table 1: Components of the positional evaluator. Parameters specify the relative strength of each feature within the shared score scale.

Term	Chess information
M_c	Weighted counts of pawns, knights, bishops, rooks, and queens.
L_c	Piece-specific square preferences, with phase-dependent tables for pawns and kings.
P_c	Advancement of passed pawns and penalties for isolated pawns.
K_c	Pawn cover and exposed files near the king, weighted by game phase and opposing heavy pieces.
C_c	Occupation and attacks on central squares and the surrounding ring.
W_c	King coordination and opposing king displacement during conversion of a material lead.

Material and placement. Write $\mathcal{P}_c(s)$ for the pieces of player c, and let $t(p)$ and $q(p)$ denote a piece's type and square. Material and placement take the form

$$M_c(s) = \sum_{p \in \mathcal{P}_c(s)} v_{t(p)}, \qquad L_c(s) = \sum_{p \in \mathcal{P}_c(s)} b_{t(p)}\big(q(p), c; \phi_c(s)\big). \tag{3}$$

The king's material weight is zero because checkmate is handled by Equation (1). Square tables use a consistent orientation for both colors.

Phase and structure. The phase variable is determined by the opponent's remaining material outside pawns and kings. If that material is $m_{\bar{c}}^{\mathrm{np}}(s)$ and the reference scale is $m_0 > 0$, then

$$\phi_c(s) = \max\left\{0, 1 - \frac{m_{\bar{c}}^{\mathrm{np}}(s)}{m_0}\right\}, \qquad b_t(q, c; \phi) = (1-\phi) b_t^{\mathrm{early}}(q, c) + \phi b_t^{\mathrm{late}}(q, c). \tag{4}$$

Nonnegative material gives $\phi_c(s) \in [0, 1]$. For piece types with a single table, the two endpoint tables coincide. This construction allows pawn advancement and king activity to gain value as the opposing force is reduced. The two players can have different phase values in an asymmetric position.

The remaining terms express structural features of the current board. A passed pawn has no opposing pawn ahead on its own or an adjacent file; its bonus grows as promotion approaches. Isolated pawns lack a friendly pawn on an adjacent file. King safety penalizes missing pawn cover and nearby files exposed to heavy pieces, with reduced weight in the endgame. Central control rewards both occupation and attacks. The conversion term becomes active with a sufficient material lead and favors coordinated king activity against an opposing king away from the center.

Evaluation perspective. The static value uses the perspective required by negamax:

$$E_\theta(s) = F_\theta(s, c(s)) - F_\theta(s, \overline{c(s)}) - \kappa\, \mathbf{1}\{c(s) \text{ is in check}\}, \tag{5}$$

where $\kappa \geq 0$ is the check penalty.

4.2 Adversarial Backups and Search Bounds

We first define a reference search with a fixed nominal horizon. Let $Q_\theta(s, \ell)$ be the tactical frontier value specified in Section 4.3. With a maximum path length $L_{\max}$, the reference value is

$$V_d(s, \ell) = \begin{cases} \tau(s, \ell), & s \in \mathcal{Z}, \\ E_\theta(s), & \ell \geq L_{\max}, \\ Q_\theta(s, \ell), & d \leq 0, \\ \max\limits_{a \in \mathcal{A}(s)} \big[-V_{d-1}(T(s, a), \ell + 1)\big], & \text{otherwise.} \end{cases} \tag{6}$$

The sign change accounts for alternating players in a zero-sum game. At a completed root search with $d \geq 1$, the corresponding decision is

$$a_d \in \arg\max_{a \in \mathcal{A}(s)} \big[-V_{d-1}(T(s, a), 1)\big]. \tag{7}$$

This maximization assumes that each opponent reply is selected to maximize the opponent's value. It therefore evaluates a candidate through the strongest response found within the examined tree.

Alpha–beta bounds. Alpha–beta search evaluates this recursion with a window $[\alpha, \beta)$. The lower threshold α tracks the best established alternative; the upper threshold β is inherited from an ancestor's alternatives. If action a_i has returned value q_i, the update is

$$\alpha_{i+1} = \max\{\alpha_i, q_i\}, \qquad \alpha_{i+1} \geq \beta \implies \text{stop expanding the remaining actions.} \tag{8}$$

A recursive negamax call uses the reversed window $[-\beta, -\alpha_i)$. A cutoff establishes a sufficient bound for an ancestor's choice. The returned quantity may therefore be a bound on a node's value.

4.3 Tactical Continuation at the Frontier

A depth limit can interrupt an exchange immediately after one side captures a piece. Static evaluation at that point can assign value to a gain that the opponent can immediately recover. Quiescence search extends the frontier through a restricted set of tactical actions.

Tactical actions and backups. For a nonterminal state, define

$$\mathcal{C}(s) = \begin{cases} \mathcal{A}(s), & c(s) \text{ is in check,} \\ \{a \in \mathcal{A}(s) : a \text{ captures or promotes}\}, & \text{otherwise.} \end{cases} \tag{9}$$

Let $g(s) = -\infty$ when the player to move is in check, and $g(s) = E_\theta(s)$ otherwise. The unpruned tactical target is

$$Q_\theta(s, \ell) = \begin{cases} \tau(s, \ell), & s \in \mathcal{Z}, \\ E_\theta(s), & \ell \geq L_{\max}, \\ \max\left\{ g(s),\ \max\limits_{a \in \mathcal{C}(s)} [-Q_\theta(T(s, a), \ell + 1)] \right\}, & \text{otherwise.} \end{cases} \tag{10}$$

We use $\max \varnothing = -\infty$. Away from check, the static score serves as the *stand-pat* candidate. In check, every legal evasion is considered and the stand-pat candidate is disabled. The path cap ensures termination even when tactical continuations include long sequences of checks.

Alpha–beta bounds also apply within this tactical tree. The stand-pat candidate is a heuristic assumption about the value of stopping local tactical analysis. Positions involving zugzwang can violate that assumption. Tactical continuation resolves only the classes of moves in Equation (9); quiet threats can remain beyond its frontier.

4.4 Selective Depth Allocation

Uniform depth assigns the same remaining horizon to actions with very different tactical consequences. The engine modifies this allocation in two ways. An extension preserves additional depth for checks and advanced pawn moves. A reduction first examines eligible late quiet moves at a shorter horizon. Eligibility excludes captures, promotions, positions already in check, and moves receiving an extension.

Depth allocation. For an action a_i in search order, write

$$d_i = d - 1 + e(s, a_i), \qquad d_i^{\text{test}} = d_i - r(s, a_i, d, i), \tag{11}$$

where $e \geq 0$ and $r \geq 0$ are bounded integer adjustments. A pathwise extension allowance limits accumulated extensions, and $L_{\max}$ bounds the total path length.

Reduced-depth probes. Let $\mathsf{S}(s, d, \alpha, \beta)$ denote the practical bounded search, with root distance and path allowances implicit. For an eligible reduced action with successor $s_i = T(s, a_i)$, a narrow-window probe computes

$$\widetilde{q}_i = -\mathsf{S}\big(s_i, d_i^{\text{test}}, -\alpha_i - \delta, -\alpha_i\big), \tag{12}$$

where $\delta > 0$ is the score resolution. If $\widetilde{q}_i > \alpha_i$, the action is searched again at depth d_i with the full current window. Other actions are searched directly at depth d_i.

The probe asks whether the action can improve the incumbent value. Its shorter horizon provides a low-cost test, but can underestimate a continuation whose benefit appears later. Re-search recovers some promising actions; a reduced action that fails the test receives no deeper verification. The practical search can consequently select a different action from the fixed-horizon rule in Equation (7).

4.5 Ordering and Transposition Reuse

Move ordering uses information from the current position and previous search work. A stored best move receives priority, captures are ranked using the values of the victim and attacker, and promotions receive a tactical preference. Quiet actions are also ranked through killer moves and a history score. Killer moves record quiet actions that caused a cutoff at the same root distance. The history score accumulates credit for quiet actions that caused cutoffs, with larger credit for greater search depth (Schaeffer, 1989). These statistics are exploration preferences within the current decision process.

Cached bounds. A transposition record stores a position key, the searched depth, the returned score, its bound type, and a preferred action. Under a compatible horizon and search context, a record with score v has one of three meanings:

$$V = v \quad \text{(exact)}, \qquad V \geq v \quad \text{(lower bound)}, \qquad V \leq v \quad \text{(upper bound)}. \tag{13}$$

An eligible exact record supplies a value directly. A lower bound at least β, or an upper bound at most α, suffices to end the current query. The stored action can guide ordering even when the value is ineligible.

The engine uses a compact key for the board position and move-count state, filters reuse at nodes with detected repetition, and accepts records whose stored depth meets the current depth request. Mate values are normalized for root distance before reuse. These rules support practical reuse; compatibility with the full history and selective-search context remains a separate requirement. In particular, an entry from a deeper horizon supplies information from a different search tree.

4.6 Opening Knowledge and Budgeted Decisions

The opening book maps covered early positions to legal candidate actions with nonnegative weights $w(s, a)$. For a relative threshold $\eta \in (0, 1]$, define

$$\mathcal{B}_\eta(s) = \left\{ a \in \mathcal{B}(s) : w(s, a) \geq \eta \max_{b \in \mathcal{B}(s)} w(s, b) \right\}. \tag{14}$$

When this set is nonempty and the state lies within the permitted opening prefix, an action is sampled uniformly from $\mathcal{B}_\eta(s)$. For other nonterminal states, the engine invokes search. The book weights are external inputs to this rule.

Iterative deepening. Search proceeds at nominal depths $1, 2, \ldots, d_{\max}$. Let t_d be the elapsed time when iteration d completes and $\hat{a}_d$ its returned action. The budgeted decision is

$$\hat{a}_B = \hat{a}_{d^\star}, \qquad d^\star = \max\{d \leq d_{\max} : t_d \leq B\}, \tag{15}$$

whenever at least one iteration completes. A legal fallback is retained for earlier interruption. The engine can also stop after finding a mate-valued result. A partially explored iteration leaves the last completed decision available.

Algorithm 1 Budgeted chess decision

Require: Nonterminal state s, time allowance B, depth limit $d_{\max}$
1: If $\mathcal{B}_\eta(s)$ is eligible, return a uniform sample from it
2: Retain a legal fallback action $\hat{a} \in \mathcal{A}(s)$
3: **for** $d = 1, \ldots, d_{\max}$ **do**
4: Run selective search at depth d within the remaining time allowance
5: If interrupted, stop and retain the previous action
6: Retain its returned action as $\hat{a}$
7: Stop if a mate-valued result is found
8: **end for**
9: **return** $\hat{a}$

An interrupted pass may contain fully evaluated subtrees. Their cached bounds remain subject to the compatibility requirements in Section 4.5. Updating such a record leaves the retained root action unchanged; the action is replaced only after a successful iteration. This preserves a completed decision while allowing reusable local work to accumulate.

5 SEARCH PROPERTIES

The value-preserving property of alpha–beta concerns a fixed search tree with compatible leaf values and reused bounds (Knuth and Moore, 1975).

Proposition 1 (Fixed-tree pruning)**.** *Consider a finite alternating game tree with fixed leaf scores from the player-to-move perspective. Alpha–beta search with a full root window returns its exhaustive negamax value, provided every reused bound is valid for the queried subtree. Move ordering can change the amount of work and the choice among tied actions.*

Proof. Negamax propagates each child value with a sign change. After any prefix of the action list, α is at least the largest established candidate. When a candidate reaches β, an ancestor already has an alternative that makes the unexamined remainder irrelevant to its maximizing choice. Pruning that remainder preserves the ancestor's backed-up value. Applying this argument inductively from the leaves to the full-window root proves the claim. □

Scope of the guarantee. The proposition applies to a tactical frontier treated as part of the fixed tree. It also applies to extensions whose allocation is fixed in advance. Late move reductions make depth depend on ordering and intermediate bounds, and cross-depth transposition reuse can change the evaluated horizon. Equation (15) therefore returns the outcome of a completed *selective* iteration.

Computational cost. For a regular tree with branching factor b and depth D, exhaustive search examines b^D leaves. With ideal move ordering and even D, alpha–beta needs $2b^{D/2} - 1$ leaves in the classical best case (Knuth and Moore, 1975). The worst-case order remains exponential. Extensions and tactical search increase the effective depth; reductions decrease work selectively. With at most H transposition records, depth-first traversal stores $O(bD + H)$ move and cache information, excluding the input game history. Wall-clock cost additionally depends on legal-move generation and evaluation.

6 DEPLOYMENT

The engine is deployed as the computer player in a local desktop chess application. The interface maintains the live game, while a separate process executes Algorithm 1. This arrangement connects a potentially long search to an interactive board and a running clock. The examples below were captured from the application launched through `game.py`; arrows, numbered pointers, and the trace plot are explanatory overlays.

6.1 FROM A LIVE POSITION TO A COMMITTED ACTION

State transfer. At the start of a computer turn, the application submits a snapshot of the position and its move history to the search process. The worker reconstructs the search state, including the history used for draw detection. Search then operates on that snapshot while the interface continues to handle events. Figure 1 identifies the information that connects the live game to this decision: legal actions, remaining time, move history, and a positional estimate.

Action validation. An interruption caused by a new game, a loaded position, or an undo invalidates the outstanding search. Each request belongs to a generation of the live game; a returned result is accepted only if that generation is still current. The application also checks that the game remains active and the action is legal before updating the board and clock. These checks keep a delayed response from being applied to a changed position. Completed search iterations can update the retained search result without changing the live board. Figure 2 summarizes this separation between computing a candidate and applying it to the game.

Clock-aware deliberation. Let B_{cap} be the configured allowance per move and $B_{\mathrm{clock}}(s)$ the allowance derived from remaining time and increment. For a timed game, the worker receives

$$B(s) = \min\{B_{\mathrm{cap}}, B_{\mathrm{clock}}(s)\}. \tag{16}$$

The clock allocation reserves time for future turns and execution overhead. The resulting allowance controls the iterative decision rule in Equation (15). Deadline and cancellation checks interrupt the current traversal; the latest completed action remains available for return. Process communication and interface updates occur outside the measured search interval.



Figure 1: The deployed chess interface, annotated to connect visible game information to the search framework. The selected knight exposes legal actions; clocks constrain deliberation; committed moves preserve game history. The optional position estimate displays the static evaluator from White's perspective, in pawn units. It is recomputed for the visible board.

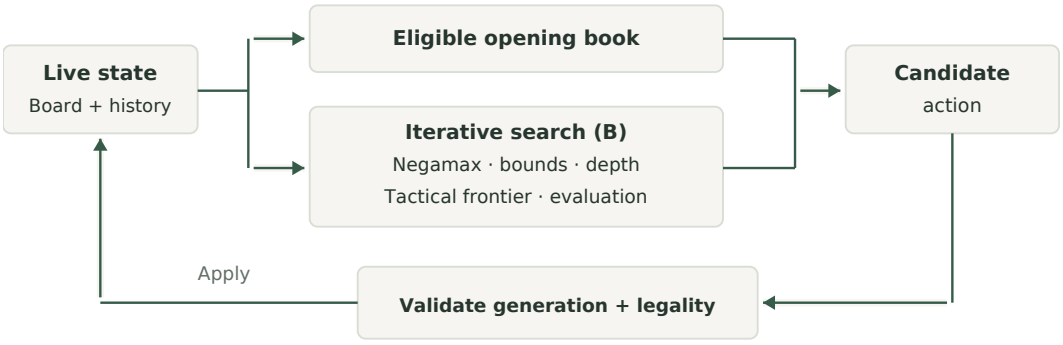


Figure 2: The deployed decision loop. A state snapshot invokes either the eligible opening rule or budgeted search. Both paths return an action through the same validation step before the live game advances. The search block combines the mechanisms formalized in Section 4.

Figure 3 shows this rule during a captured computer turn. The fifth iteration completed at 1.282 s. Search continued until the 1.800 s allowance was reached, and the returned action remained the fifth iteration's dxc6. The trace records nominal depth: extensions, reductions, and tactical continuation can produce paths of different lengths within an iteration. Its timestamps describe this execution on the host machine.

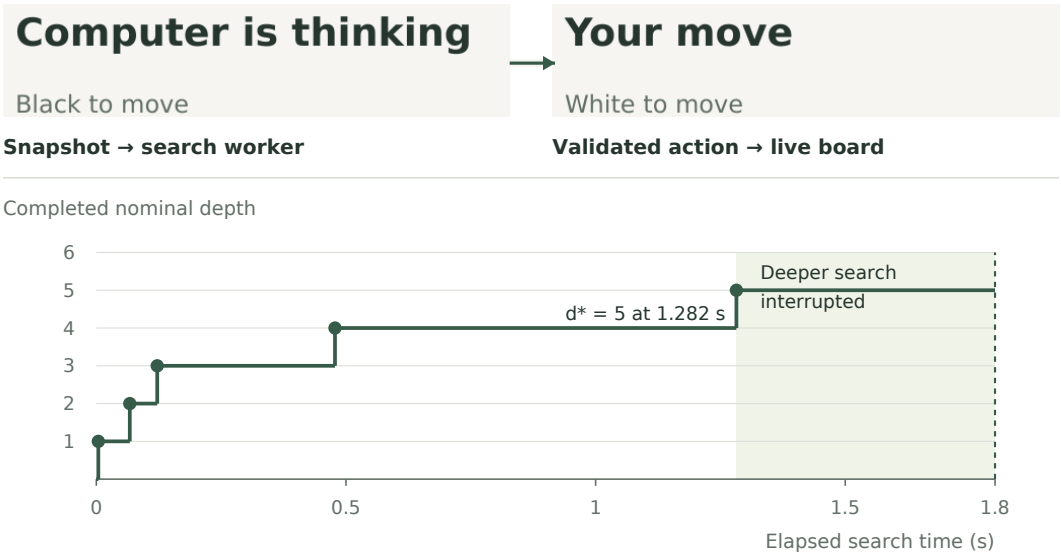
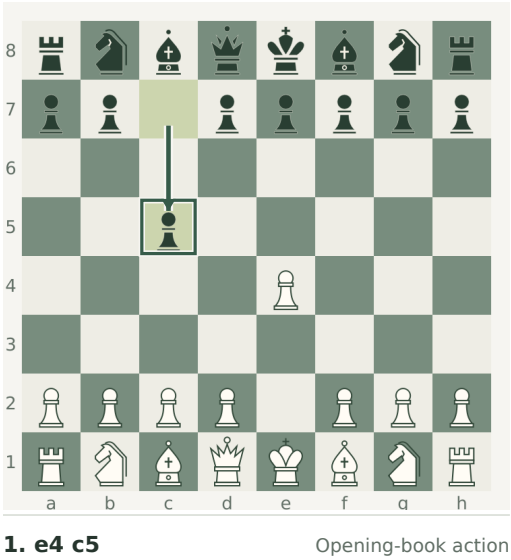


Figure 3: A captured search with the opening book disabled. Interface excerpts show the transition from computation to the human turn. The plot reconstructs completed iterations from worker messages, illustrating Equation (15). The shaded interval contains unfinished deeper search. Each completed iteration selected dxc6, the action subsequently applied to the live game.

Across the completed iterations, dxc6 remains the selected action while its backed-up score changes from -1.79 to -0.60 pawn units. These scores use Black's perspective at the search root and summarize different examined continuations. The worker reports $12{,}000$ node visits at completion of the fifth iteration and $17{,}394$ at interruption. The additional local analysis leaves the completed root decision available, as required by Algorithm 1. The optional board estimate in Figure 1 uses the current position and White's perspective, following the static evaluation in Equation (5).

6.2 Search Mechanisms in Play



Opening knowledge. After the human plays e4, the covered position invokes the book rule in Equation (14). In this execution, the eligible set contains only c5, so the computer selects that action directly. The arrow traces c7–c5. Book weights determine eligibility through the relative threshold; sampling is uniform over the surviving actions. Each eligible action has probability $1/|\mathcal{B}_\eta(s)|$ at the covered state. At other covered positions, several entries may survive the threshold, so repeated games can select different opening continuations.

Book selection supplies an action without a completed root iteration or a backed-up search value. When the permitted prefix ends or no eligible entry exists, the worker invokes iterative deepening. The returned candidate passes the shared validation step in Figure 2; acceptance updates the board and starts the next player's clock.

Figure 4: An opening-book decision after 1. e4. The overlay traces c7–c5 on the live board. Book and search actions undergo the same validation before application.

An unsettled tactical frontier. Figure 5 follows a human capture and the computer's reply in the position used for Figure 3. After Bxc6, White has temporarily gained a knight. The reply dxc6 removes the capturing bishop, leaving the material difference close to its initial value. A frontier placed between these two actions would expose an incomplete exchange to static evaluation. Equation (10) addresses this situation by retaining legal captures in the tactical action set; a search reaching the intermediate position at its nominal horizon can therefore examine the recapture. At this intermediate position Black is not in check. Its static score remains a stand-pat candidate, and dxc6 belongs to $\mathcal{C}(s)$. The tactical backup compares the static value with the negated successor value of each retained action.

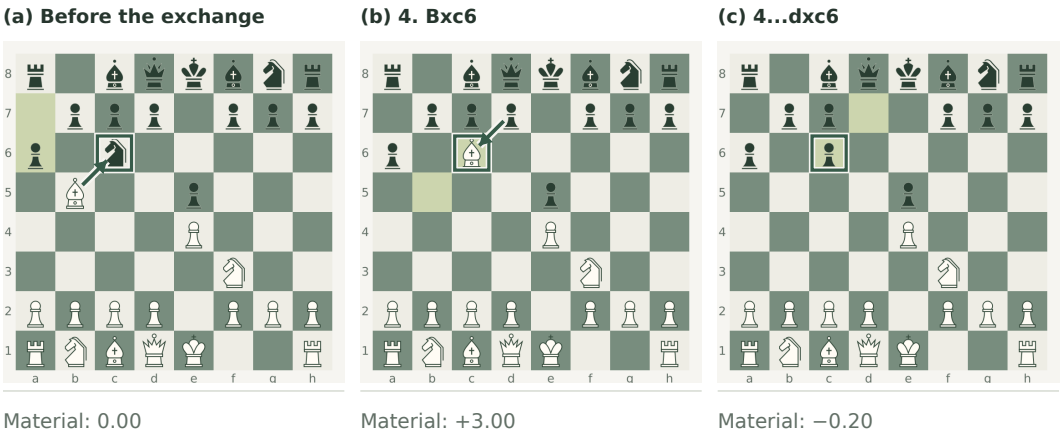
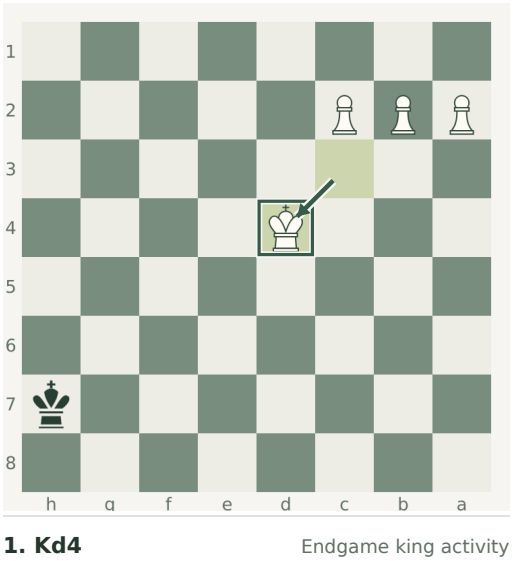


Figure 5: A captured exchange after 1. e4 e5 2. Nf3 Nc6 3. Bb5 a6. Arrows mark the human's 4. Bxc6 and the engine's 4...dxc6. Outlines follow the exchange square. Material differences are shown from White's perspective in pawn units using the engine's piece weights; they exclude positional terms. The sequence illustrates the frontier issue addressed by tactical continuation.



Phase-dependent evaluation. With only kings and White's three pawns present, both phase variables in Equation (4) equal one. The late king table is fully active, and White's material lead activates the conversion term W_c. The engine completes a depth-three search and plays Kd4, leaving all three pawns in place. The arrow identifies the king's advance on a board oriented toward the human playing Black.

At this transition, material and pawn structure remain unchanged, while king placement, central control, and conversion features change. Deeper continuations also examine pawn advances and opposing king replies. The phase and feature values are recomputed along those paths and combined through negamax backups. The selected king move connects these late positional preferences to a choice evaluated through the opponent's replies.

Figure 6: King activity in a captured endgame decision. White moves from c3 to d4 while preserving its three pawns. The overlay identifies the action evaluated under the late-game phase; the position illustrates the operating context of the phase and conversion terms.

Together, these cases make three choices in Algorithm 1 visible: selecting a covered opening action, resolving a tactical sequence, and searching with phase-dependent frontier values. They document the execution of the integrated engine. The contribution of each component to move quality remains a question for controlled comparisons at matched budgets.

7 Discussion and Limitations

The decision procedure has three main approximation limits. First, fixed positional features provide incomplete information about long-term chess outcomes. Phase-dependent tables and structural bonuses encode selected preferences, and their weights require empirical assessment. Heuristic scores alone establish no playing strength or calibration.

Second, finite and selective horizons can miss decisive continuations. Quiescence examines captures, promotions, and check evasions, while quiet tactical ideas may remain unresolved. Stand-pat evaluation assumes that a nonchecked player can maintain the assessed value. Reduced-depth probes can discard a useful action before its benefit becomes visible, and reaching the absolute path cap terminates tactical analysis even in an unsettled position.

Third, reuse depends on information that a compact position key only partially represents. Distinct repetition histories and extension contexts can produce different search values for the same keyed position. Repetition filtering reduces some problematic reuse, but complete compatibility requires those contexts to agree. Accepting a deeper record also changes the finite-horizon query. In addition, the search treats its recognized move-count and repetition draw conditions as terminal; an explicit player decision about claiming a draw is outside this model.

A completed deeper iteration supplies a new decision, with no guarantee that playing strength improves monotonically with depth. Empirical claims about strength, efficiency, and component interactions require controlled comparisons at matched resource budgets. The deployment trace establishes the sequence of completed decisions in one execution; the annotated positions document their application to the live game.

8 Conclusion

We have described a budgeted chess decision procedure built from positional evaluation and selective adversarial search. Negamax specifies the backup rule; alpha–beta uses bounds to avoid unnecessary expansion on a fixed tree. Tactical continuation, depth adjustments, ordering, and transposition reuse determine which continuations receive computation. Iterative deepening connects these components to an interruptible decision process, and the opening book supplies actions in covered early positions. The desktop deployment connects these decisions to a live game through state snapshots, clock-aware allowances, and validation before an action is applied. The formulation and captured execution make both the search assumptions and their operational consequences explicit. Controlled comparisons remain necessary to assess playing strength and the value of individual enhancements.

References

Campbell, M., Hoane, Jr., A. J., and Hsu, F.-h. (2002). Deep Blue. *Artificial Intelligence*, 134(1–2):57–83.

Grill, J.-B., Altché, F., Tang, Y., Hubert, T., Valko, M., Antonoglou, I., and Munos, R. (2020). Monte-Carlo tree search as regularized policy optimization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3769–3778. PMLR.

Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326.

Schaeffer, J. (1989). The history heuristic and alpha-beta search enhancements in practice. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(11):1203–1212.

Shannon, C. E. (1950). Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144.